

CENG 467 – Natural Language Understanding and Generation

Take-Home Midterm Examination Report

Yusuf Furkan Aktay
Department of Computer Engineering
Izmir Institute of Technology

April 2026

Abstract

This report presents a comprehensive comparative study of five core NLP tasks: text classification, named entity recognition (NER), text summarization, machine translation, and language modeling. For each task, we implement and evaluate multiple representative model families – spanning classical (TF-IDF, n-gram), neural (BiLSTM, LSTM-LM, Seq2Seq with attention), and transformer-based (DistilBERT, BART, opus-mt) approaches – on standard benchmark datasets. All experiments are reproducible with a fixed random seed and were carried out on Google Colab Free’s T4 GPU under realistic compute constraints. Across all five tasks, transformer-based contextual models consistently outperform classical and shallow neural baselines, but the magnitude of improvement and the metric-specific behavior (e.g., BLEU vs. METEOR vs. BERTScore) reveal nuanced trade-offs between fluency, factual fidelity, and computational cost. We complement quantitative results with detailed error and qualitative analysis to characterize each model’s behavior and limitations.

1 Introduction and Experimental Protocol

This take-home midterm covers five canonical tasks in Natural Language Understanding and Generation. Each task constitutes an independent experimental study, and all share a common reproducibility protocol described below.

1.1 Reproducibility

A single random seed (**SEED** = 42) is set across Python’s **random**, NumPy, PyTorch (CPU and CUDA), and the HuggingFace **Trainer** interface. All deterministic operations are enabled where supported. Datasets are loaded from canonical mirrors (HuggingFace Datasets where available; otherwise from the original GitHub repositories), with fixed splits. The test set in every task is used *only once* for final evaluation.

1.2 Compute Environment

All experiments were carried out on Google Colab’s free tier with the following specification:

- Hardware: NVIDIA T4 GPU (15 GB VRAM)
- Software: Python 3.12, PyTorch 2.x, transformers 4.5x (current), datasets 3.x, scikit-learn 1.4

The free-tier compute budget imposes practical constraints (runtime caps, limited parallel workers). We made several principled engineering decisions to remain within budget while preserving experimental validity:

- Where the exam allows subsets (Q3), we use a 100-article test sample.
- Where deep neural training is required (Q1, Q2, Q4, Q5), we cap training corpora and epochs to fit within ~ 10 minutes per task while preserving the qualitative trends reported in the literature.
- For pre-trained models intended for inference comparison (Q3 BART, Q4 opus-mt), we use them *without* further fine-tuning – a standard practice in compute-aware comparative research.

These decisions are documented per task. We emphasize that the *relative* comparisons between models – which is the core pedagogical objective of this exam – remain valid under these constraints.

1.3 Repository

All notebooks, results files, and this report are publicly available at:

https://github.com/FurkanAktay/CENG467_Midterm

2 Q1: Representation Learning in Text Classification

2.1 Problem Formulation and Dataset

We compare three representation paradigms – *sparse*, *dense*, and *contextual* – on binary sentiment classification. The dataset is SST-2 (Stanford Sentiment Treebank, binary version) from the GLUE benchmark, comprising single-sentence movie reviews labeled positive/negative. Following standard GLUE evaluation practice (where the official test labels are private), we treat the GLUE *validation* split as our held-out test set, and create a fresh 90/10 train/dev split from the original training set.

Table 1: SST-2 dataset statistics (after our 90/10 split).

Split	Examples
Train	60,614
Dev (held out from training)	6,735
Test (= GLUE validation)	872

2.2 Preprocessing and Tokenization

For the sparse classical model we compare two tokenization strategies:

1. **Simple:** lowercase + whitespace split.
2. **NLTK + stopword removal:** lowercase, regex-strip non-alphabetic characters, NLTK `word_tokenize`, drop English stopwords and 1-character tokens.

The neural BiLSTM uses simple whitespace tokenization with a frequency-pruned vocabulary (min freq 2, top 20K words). DistilBERT uses its native WordPiece tokenizer (`distilbert-base-uncased`).

2.3 Models and Training

TF-IDF + Logistic Regression. TF-IDF features with unigrams and bigrams, `max_features=50,000`, `min_df=2`; logistic regression with $C = 1.0$, `max_iter=1000`.

BiLSTM. 128-dim trainable embeddings, single-layer bidirectional LSTM with 128 hidden units, mean-pool over time, 0.5 dropout, linear classifier. Trained 5 epochs with Adam ($\text{lr}=10^{-3}$), batch size 64, max sequence length 50.

DistilBERT. The pre-trained `distilbert-base-uncased` model is fine-tuned with a sequence classification head. Trained 2 epochs with AdamW ($\text{lr}=2 \times 10^{-5}$, weight decay 0.01), batch size 32, max sequence length 64.

2.4 Results

Table 2: SST-2 test results across three representation paradigms.

Model	Accuracy	Macro-F1	Time (s)
TF-IDF + Logistic Regression (sparse)	0.8177	0.8161	14.1
BiLSTM (dense)	0.8050	0.8045	21.6
DistilBERT (contextual)	0.9037	0.9035	667.7

A short tokenization study showed simple whitespace tokenization slightly outperformed NLTK+stopword-removal for TF-IDF; the simple variant was retained as the final classical baseline.

2.5 Error Analysis

DistilBERT misclassified 84/872 ($\approx 9.6\%$) of test examples. We inspected the first five errors and identified three recurring failure patterns:

- (1) **Concessive / contrastive constructions.** The example *“we root for (clara and paul), even like them, though perhaps it’s an emotion closer to pity.”* is labeled positive but classified negative by all three models. The clause flow shifts polarity multiple times (root for $\rightarrow$ like $\rightarrow$ pity), and shallow surface cues weight the negative-loaded final clause.
- (2) **Implicit comparison / sarcasm.** *“holden caulfield did it better.”* – gold label negative, all three models predict positive. The sentence is a sarcastic put-down (the current work is worse than *Catcher in the Rye*), but contains no explicit negative lexical cue.
- (3) **Indirect or literary phrasing.** *“the script kicks in, and mr. hartley’s distended pace and foot-dragging rhythms follow.”* – gold negative, BERT predicts positive. The negative meaning is carried by “distended,” “foot-dragging,” i.e., domain-specific cinematic critique vocabulary; these are likely under-represented in BERT’s pretraining for the polarity task.

These errors are characteristic of SST-2 and are well-documented in the literature on the limits of bag-of-words and even contextual classifiers for sentiment with figurative language.

2.6 Discussion

DistilBERT improves macro-F1 by ~ 9 absolute points over both classical and shallow neural baselines, confirming the value of contextual representations. Interestingly, the simple TF-IDF baseline marginally outperforms BiLSTM here – a finding consistent with the observation that on short, lexically-driven sentences (SST-2 averages ~ 10 words), n-gram features can already capture most sentiment-bearing signal, and a small from-scratch BiLSTM does not have enough data or capacity to beat them. The cost is large: DistilBERT requires roughly $50\times$ the training time of TF-IDF for ≈ 9 -point F1 improvement – a useful concrete data point for the cost-quality trade-off in production.

3 Q2: Named Entity Recognition

3.1 Problem Formulation and Dataset

We use the CoNLL-2003 English NER benchmark, derived from Reuters newswire. Four entity types are annotated using the BIO scheme: **PER** (person), **LOC** (location), **ORG** (organization), and **MISC** (miscellaneous). Splits: 14,041 train / 3,250 dev / 3,453 test sentences.

Owing to the deprecation of script-based dataset loaders in the latest `datasets` library, we obtain CoNLL-2003 from the `tomaarsen/conll2003` parquet mirror, which preserves the original tags and splits.

3.2 Models

BiLSTM-CRF (hybrid). A 100-dim trainable embedding layer, a bidirectional LSTM with 128 hidden units (each direction), 0.3 dropout, and a Conditional Random Field decoding layer (`pytorch-crf`). The CRF enforces BIO consistency via learned transition scores. Trained 5 epochs with Adam ($\text{lr}=10^{-3}$), batch size 32, max sequence length 80.

DistilBERT-NER (transformer). Token classification head on top of `distilbert-base-uncased`. We follow the standard subword-alignment convention: only the first subword of each original word receives the gold tag, and continuation subwords receive the ignore-index -100 . Owing to runtime constraints, training was performed on a 5,000-sentence subset of CoNLL-2003 (full training set has 14,041 sentences) for 2 epochs with AdamW ($\text{lr}=3 \times 10^{-5}$, weight decay 0.01), batch size 16, max sequence length 128. We discuss this choice further in Section 2.5.

3.3 Results

We use entity-level Precision, Recall, and F1 from `segeval`, the standard library for CoNLL-style evaluation.

Table 3: CoNLL-2003 test results.

Model	Precision	Recall	F1	Time (s)
BiLSTM-CRF	0.7411	0.5808	0.6512	240.5
DistilBERT-NER (5K subset)	0.8916	0.9014	0.8965	218.7

Note: BiLSTM-CRF numbers correspond to the run that produced the saved `q2_full.json`; absolute values may vary slightly across reruns due to GPU non-determinism.

Table 4: DistilBERT-NER per-entity breakdown on the test set.

Entity	Precision	Recall	F1	Support
LOC	0.8912	0.9335	0.9119	1668
MISC	0.7868	0.7991	0.7929	702
ORG	0.8555	0.8627	0.8591	1661
PER	0.9778	0.9524	0.9649	1617
micro avg	0.8916	0.9014	0.8965	5648
macro avg	0.8778	0.8869	0.8822	5648
weighted avg	0.8925	0.9014	0.8968	5648

3.4 Error Analysis

We inspected sentences where DistilBERT predictions differ from gold annotations. Three error categories dominate:

(1) Boundary errors with multi-token entities. In the sentence “*Cuttitta announced his retirement after the 1995 World Cup...*”, the gold entity is B-MISC I-MISC I-MISC for “1995 World Cup” (a single MISC entity for the event). DistilBERT tags “1995” as 0 and starts the entity at “World”, producing only “World Cup”. This is a classical boundary error, particularly common when an entity is initiated by a numeric token.

(2) Domain shift / entity-type confusion. CoNLL-2003 contains cricket scorecards with idiosyncratic format such as “*Inzamamul Haq st Germon b Astle 2*”. Here “Astle” is a player name (PER), but DistilBERT tags it as B-ORG – an *entity-type confusion* driven by domain-specific token patterns the pre-training corpus rarely covers. The BiLSTM-CRF makes related but different mistakes (mistagging the entire scorecard as ORG), reflecting that neither model has good coverage of this micro-domain.

(3) Annotation ambiguity / world knowledge. In “... *CHINA IN SURPRISE DEFEAT*”, the gold label for “CHINA” is B-PER – referring to a Chinese player whose nickname is the country – whereas DistilBERT and BiLSTM both tag it as B-LOC. Resolving this requires sports-domain world knowledge that neither model possesses.

(4) MISC weakness. The per-entity table shows MISC F1 is the lowest at 0.79, ~15 points below PER. MISC is a notoriously loose category in CoNLL-2003 (covering nationalities, events, languages, titles), and this gap is consistent with all published benchmark results.

3.5 Discussion

DistilBERT-NER outperforms the BiLSTM-CRF baseline by ~25 F1 points despite training on only 5,000 sentences. The advantage stems from contextual embeddings pre-trained on a large corpus: subword tokenization handles unseen surface forms (e.g., “Inzamamul” as multiple subwords whose contextual embeddings are still informative), and bidirectional attention captures longer-range dependencies than a single-layer BiLSTM can. The CRF layer in the BiLSTM model still adds value (it enforces BIO consistency), but the underlying token representations are simply too weak to compete.

We emphasize that even with 5,000 sentences (~36% of full training data), DistilBERT achieves F1=0.8965, which is within ~2 points of the full-data result reported in literature (~0.91). This suggests strong sample efficiency from pre-training – a finding with practical importance in low-resource NER settings.

4 Q3: Text Summarization

4.1 Problem Formulation and Dataset

We compare *extractive* and *abstractive* summarization on the CNN/DailyMail v3.0.0 corpus, a benchmark of news articles paired with multi-sentence reference summaries (“highlights”). Following the exam’s allowance for subsets (and to remain within Colab’s runtime), we evaluate on a fixed 100-article test sample drawn with seed 42 from the full 11,490-article test split.

Table 5: CNN/DailyMail test subset statistics.

Statistic	Value
Number of articles	100
Avg. article length (chars)	~3,900
Avg. reference length (chars)	~280

4.2 Models

LexRank (extractive). LexRank builds a graph whose nodes are article sentences and whose edges are weighted by TF-IDF cosine similarity, then runs PageRank to identify the most central sentences. We extract the top-3 sentences per article (matching the typical 3-sentence reference structure of CNN/DM). We use the implementation from the `sumy` library.

BART-CNN (abstractive). We use `facebook/bart-large-cnn`, a BART encoder-decoder already fine-tuned on CNN/DailyMail. Generation uses beam search with 4 beams, length penalty 2.0, min length 30, max length 130, and early stopping. *No further fine-tuning is performed.*

4.3 Evaluation Metrics

We report a comprehensive metric suite covering n-gram overlap, alignment, and semantic similarity:

- **ROUGE-1, ROUGE-2, ROUGE-L** (`rouge_score`, with stemming) – standard summarization metrics measuring unigram, bigram, and longest-common-subsequence F-scores.
- **BLEU** (`sacrebleu`, corpus-level) – translation-style precision metric with brevity penalty.
- **METEOR** (NLTK) – alignment-based metric matching synonyms and stems.
- **BERTScore-F1** (`bert-score`, `lang=en`) – contextual semantic similarity using BERT embeddings.

4.4 Results

Table 6: Summarization results on a 100-article CNN/DailyMail test subset.

Model	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore-F1
LexRank (extractive)	0.3445	0.1239	0.2196	0.0856	0.3225	0.8624
BART-CNN (abstractive)	0.4316	0.2041	0.3022	0.1588	0.3892	0.8813

BART improves all six metrics; the largest absolute gains are on ROUGE-2 (+8.0 points) and BLEU (+7.3 points), reflecting BART’s far stronger bigram and exact-phrase agreement with references. The smallest gain is on BERTScore-F1 (+1.9 points), suggesting that LexRank, despite extracting verbatim sentences, captures most of the *semantic* content that BERT can detect.

4.5 Qualitative Analysis

We manually inspect three representative pairs (full transcripts in `results/q3_examples.json`).

Example 1 – factual hallucination by BART. The article is by CNN’s Dr. Sanjay Gupta on the medical-marijuana movement. The reference: “*CNN’s Dr. Sanjay Gupta says we should legalize medical marijuana now...*”. BART produced: “*CNN’s John Sutter says he sees signs of a medical marijuana revolution... Sutter: ‘Weed 3’ is eyewitnesses to a revolution...*”. BART invents a non-existent author (“John Sutter”) and a malformed phrase (“eyewitnesses to a revolution”).

LexRank, by contrast, copies actual article sentences and remains factually faithful. This is the canonical *abstractive hallucination* failure mode.

Example 2 – coverage vs. fluency. An article on a Memphis child posting gang-life content on Twitter. LexRank copies four full sentences from the article, retaining factual accuracy at the cost of length and flow. BART produces a more compact, fluent summary that includes a paraphrased subset of the tweets. Here, the better choice depends on user need: fidelity (LexRank) versus readability (BART).

Example 3 – factual addition. An article on Governor Christie’s town hall debate. BART includes the figure “\$225 million” for the ExxonMobil settlement – a number not present in the reference summary. Whether this is a factual addition (helpful) or a hallucination (harmful) depends on whether the figure appears verbatim in the source article (we did not verify each token).

4.6 Discussion

The results illustrate three trade-offs:

Quality vs. cost. BART runs $\sim 22\times$ slower than LexRank (190.2s vs. 8.8s for 100 articles, both on T4). For high-throughput pipelines (e.g., real-time news summarization at scale), this gap is substantial.

Fluency vs. faithfulness. BART produces grammatically smoother, more compressed summaries, but is susceptible to hallucination (Example 1). LexRank cannot hallucinate by construction (it can only copy), making it the safer choice when factual fidelity is critical (e.g., medical, legal).

Metric coverage. BERTScore-F1 detects only a 1.9-point gap between the two methods, whereas ROUGE-2 detects an 8.0-point gap. BERTScore is largely insensitive to phrasing differences when the underlying semantic content is similar – this is a feature for paraphrase tolerance, but a limitation for evaluating faithfulness, since hallucinated content with similar embeddings can score highly. *No single metric should be relied on alone.*

5 Q4: Machine Translation

5.1 Problem Formulation and Dataset

We study English $\rightarrow$ German translation on the Multi30k dataset, an extension of Flickr30k where each image caption is paired with a German translation. Splits: 29,000 train / 1,014 valid / 1,000 test (`test_2016_flickr`). The data is loaded from the `bentrevett/multi30k` HuggingFace mirror, which contains the canonical Multi30k Task 1 splits in parquet format.

5.2 Models

Seq2Seq with Bahdanau Attention (custom, trained from scratch). A standard encoder-decoder with attention:

- *Encoder:* 128-dim trainable English embeddings $\rightarrow$ bidirectional GRU with 256 hidden units (each direction). The final hidden state is projected through $\tanh$ into a single 256-dim vector that initializes the decoder.
- *Attention:* additive (Bahdanau) attention – the decoder state attends over all encoder outputs via a learned MLP scoring function.
- *Decoder:* 128-dim trainable German embeddings; at each step, the decoder consumes the previous token, the attention context, and the previous hidden state, and emits a softmax over the German vocabulary.

The model has separate English and German vocabularies (frequency-pruned at min freq 2, capped at 10K each) with `<sos>`, `<eos>`, `<pad>`, `<unk>` special tokens. Trained 8 epochs with Adam ($\text{lr}=10^{-3}$), batch size 64, teacher-forcing ratio 0.5, gradient clipping at 1.0.

Helsinki-NLP/opus-mt-en-de (pre-trained Transformer). The OPUS-MT model is a Transformer encoder-decoder trained on the OPUS multilingual corpus. We use it for inference only (beam search with 4 beams, max length 128, early stopping). *No fine-tuning.*

5.3 Evaluation Metrics

We use a four-metric suite emphasizing the morphological richness of German:

- **BLEU** (sacrebleu) – corpus-level n-gram precision with brevity penalty.
- **METEOR** (NLTK) – alignment-based with synonymy and stemming.
- **ChrF** (sacrebleu) – character n-gram F-score; particularly informative for German, which inflects and compounds heavily.
- **BERTScore-F1** (lang=de) – multilingual BERT semantic similarity.

5.4 Results

Table 7: Multi30k EN→DE translation results.

Model	BLEU	METEOR	ChrF	BERTScore-F1	Time (s)
Seq2Seq + Attention (from scratch)	0.0438	0.5495	0.4102	0.7603	395.6
Transformer (opus-mt, pre-trained)	0.3637	0.6655	0.6423	0.8970	16.2

5.5 Qualitative Analysis

Example 1.

SRC: *A man in an orange hat starring at something.*

REF: *Ein Mann mit einem orangefarbenen Hut, der etwas anstarrt.*

SEQ2SEQ: *ein mann mit einem orangen hut auf etwas .*

TRANSFORMER: *Ein Mann in einem orangenen Hut, mit etwas.*

Both models capture the core noun phrase. Seq2Seq omits the relative clause “der etwas anstarrt” (*who is staring at something*) entirely; the Transformer also drops the staring action but produces grammatical German.

Example 2 – OOV failure.

SRC: *A Boston Terrier is running on lush green grass in front of a white fence.*

REF: *Ein Boston Terrier läuft über saftig-grünes Gras vor einem weißen Zaun.*

SEQ2SEQ: *ein <unk> läuft auf dem <unk> vor einem weißen zaun .*

TRANSFORMER: *Ein Boston Terrier läuft auf üppig grünem Gras vor einem weißen Zaun.*

The Seq2Seq model emits `<unk>` for “Boston Terrier” and “lush green grass” because these tokens fall outside its frequency-pruned 10K vocabulary. The Transformer’s SentencePiece subword tokenizer has no such vocabulary cliff and translates everything correctly. This is the canonical advantage of subword tokenization for low-resource MT.

Example 3 – collapse to repetition.

SRC: *A girl in karate uniform breaking a stick with a front kick.*

REF: *Ein Mädchen in einem Karateanzug bricht ein Brett mit einem Tritt.*

SEQ2SEQ: *ein mädchen gekleidetes mädchen macht einen einen mit mit einem fußtritt tritt .*

TRANSFORMER: *Ein Mädchen in Karate-Uniform bricht einen Stock mit einem Front-Kick.*

The Seq2Seq output exhibits autoregressive degeneration – repeated tokens (“einen einen”, “mit mit”, “tritt” appearing twice). This is a known pathology of small Seq2Seq models lacking sufficient capacity or regularization. The Transformer produces a near-reference-quality translation.

5.6 Discussion

The metric profile is highly informative.

BLEU is harshest on Seq2Seq. BLEU rewards exact n-gram matches; the Seq2Seq model – with its small vocabulary, frequent `<unk>` tokens, and reordering errors – collects almost no credit (0.04). The Transformer benefits from a far larger vocabulary and pre-training (0.36).

METEOR is more forgiving. METEOR’s synonym/stem matching gives the Seq2Seq partial credit for near-misses. The gap between the two models on METEOR (0.55 vs. 0.67) is much smaller than on BLEU.

ChrF rewards morphology. German’s compounding and inflection mean character-level overlap is more informative than word-level overlap. ChrF places the Transformer 23 points above Seq2Seq – consistent with BLEU’s ranking but at higher absolute values.

BERTScore reveals semantic similarity. Even when the surface forms diverge greatly, the Seq2Seq output remains semantically related to the reference (BERTScore 0.76 vs. 0.90 for the Transformer). The 14-point gap reflects the Transformer’s fluency advantage in addition to lexical correctness.

Architecture trade-offs. Our small Seq2Seq is trained from scratch on 29K sentence pairs in ~6.5 minutes; the pre-trained Transformer is much larger and was trained for orders of magnitude more data and compute. The comparison therefore reflects the value of pre-training as much as the choice of architecture. A from-scratch Transformer of similar size to our Seq2Seq would likely fall between these two – subword tokenization alone would close most of the OOV gap (Example 2).

6 Q5: Language Modeling

6.1 Problem Formulation and Dataset

We compare a classical n-gram model and an LSTM-based neural language model on the WikiText-2 raw corpus, a curated Wikipedia benchmark that retains original case, punctuation, and numbers (in contrast to Penn Treebank’s heavily preprocessed form). The dataset is loaded from `Salesforce/wikitext (wikitext-2-raw-v1)`. Sentences are obtained by line-splitting and lowercased whitespace tokenization.

6.2 Models

Trigram with Laplace Smoothing. We use NLTK’s Laplace (add-1 smoothed) trigram language model, fitted via the standard `padded_everygram_pipeline`. *Note on smoothing choice:* we initially attempted Kneser-Ney interpolated smoothing (NLTK’s `KneserNeyInterpolated`); however, the pure-Python implementation proved computationally prohibitive on the WikiText-2 training set within the Colab Free runtime. We therefore use Laplace smoothing as a tractable alternative, with the understanding that the n-gram model serves here as a baseline against which neural models are compared, not as a state-of-the-art benchmark in itself. Trained on a 10,000-sentence subset of the WikiText-2 training split.

LSTM Language Model. A 2-layer LSTM with 128-dim trainable embeddings, 256 hidden units per layer, and 0.3 dropout on inputs, between layers, and on outputs. We follow the standard truncated-BPTT training recipe: concatenate all training sentences (with `<bos>/<eos>` markers) into one long stream, partition into 32 parallel streams (batch dimension), and train on contiguous

BPTT-length windows (BPTT=25). Trained 3 epochs with Adam (lr= 10^{-3}), gradient clipping at 0.5. Vocabulary capped at 20K (min freq 2). Trained on an 8,000-sentence subset to fit Colab’s runtime budget.

6.3 Results

Table 8: WikiText-2 language modeling results.

Model	Test Perplexity	Train Time (s)
Trigram (Laplace, add-1)	19,820.30	12.2
LSTM-LM (2-layer, 256H)	272.07	39.0

The LSTM-LM achieves a perplexity that is roughly $73\times$ lower than the smoothed trigram. Both numbers are higher in absolute terms than the literature standard for full-corpus, fully-trained models on WikiText-2 (e.g., Merity et al. 2018 report ~ 66 perplexity for AWD-LSTM with extensive tuning), reflecting our subset and runtime constraints. The *relative* gap between count-based and neural models, however, is consistent with the literature.

6.4 Discussion

Why is the trigram so weak? Laplace smoothing is provably suboptimal: it assigns identical probability mass to every unseen trigram regardless of its constituent unigrams or bigrams. On WikiText-2, with a vocabulary of $\sim 33K$ types and a long tail of rare words, the vast majority of trigrams encountered at test time were not seen during training, and Laplace simply returns a uniform-ish small probability for all of them, yielding extremely high perplexity. Kneser-Ney’s continuation probabilities (which we could not run within budget) are designed precisely to address this; the gap between Laplace and Kneser-Ney on this dataset is typically large (often more than $10\times$).

Why is the LSTM-LM strong? The LSTM learns dense distributed word representations that generalize across rare contexts: rather than treating every unseen trigram as equally surprising, it estimates likelihood using continuous similarities to seen contexts. This is the canonical advantage of neural over count-based language modeling.

Trade-offs. The trigram trains in 12 seconds on CPU and uses essentially no GPU memory; the LSTM requires GPU and an order of magnitude more training time. For perplexity-driven applications the LSTM is dominant; for very fast on-device inference with no parameter budget, smoothed n-grams remain useful.

7 Conclusions

Across all five tasks, the same overarching pattern emerges: *contextual representations from large pre-trained models consistently outperform classical and shallow neural baselines, often by a wide margin.* However, the magnitude of the gap and the metric-specific behavior reveal task-dependent nuances:

- **Q1 (classification):** DistilBERT improves macro-F1 by ~ 9 points over both TF-IDF and BiLSTM, but the gap between TF-IDF and BiLSTM is small (≤ 1 point), indicating that for short, lexically-driven texts a strong sparse baseline is hard to beat without contextual pretraining.

- **Q2 (NER):** DistilBERT-NER beats BiLSTM-CRF by ~ 25 F1 points even with only 5K training sentences, demonstrating strong sample efficiency from pre-training. MISC remains the hardest entity type.
- **Q3 (summarization):** BART improves all metrics over LexRank, but is $\sim 22\times$ slower and is susceptible to factual hallucination – our Example 1 shows BART inventing a non-existent author. Extractive methods remain valuable when fidelity is paramount.
- **Q4 (translation):** OPUS-MT outperforms our from-scratch Seq2Seq dramatically, with the largest gap on BLEU (+0.32). BLEU vs. METEOR vs. ChrF vs. BERTScore reveal genuinely different aspects of translation quality, and no single metric tells the whole story.
- **Q5 (language modeling):** The LSTM-LM achieves $\sim 73\times$ lower perplexity than a Laplace-smoothed trigram, illustrating the value of dense neural representations for sequence likelihood modeling.

A second, more nuanced finding is that *evaluation metric choice matters as much as model choice*. Q3’s BERTScore vs. ROUGE-2 gap, Q4’s BLEU vs. METEOR gap, and the documented hallucination in BART’s Example 1 all point to the same lesson: surface-overlap metrics, semantic-similarity metrics, and faithfulness checks measure complementary properties, and rigorous NLG evaluation requires reporting all of them.

References

1. Devlin, J. et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *NAACL*, 2019.
2. Sanh, V. et al. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.” *NeurIPS Workshop on Energy Efficient ML*, 2019.
3. Lewis, M. et al. “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.” *ACL*, 2020.
4. Tiedemann, J. and Thottingal, S. “OPUS-MT: Building open translation services for the World.” *EAMT*, 2020.
5. Erkan, G. and Radev, D. “LexRank: Graph-based Lexical Centrality as Saliency in Text Summarization.” *JAIR*, 2004.
6. Bahdanau, D. et al. “Neural Machine Translation by Jointly Learning to Align and Translate.” *ICLR*, 2015.
7. Merity, S. et al. “Regularizing and Optimizing LSTM Language Models.” *ICLR*, 2018.
8. Tjong Kim Sang, E.F. and De Meulder, F. “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition.” *CoNLL*, 2003.
9. Socher, R. et al. “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank.” *EMNLP*, 2013.
10. Hermann, K.M. et al. “Teaching Machines to Read and Comprehend.” *NeurIPS*, 2015. (CNN/DailyMail).
11. Elliott, D. et al. “Multi30K: Multilingual English-German Image Descriptions.” *Workshop on Vision and Language*, 2016.
12. Lin, C.-Y. “ROUGE: A Package for Automatic Evaluation of Summaries.” *ACL Workshop*, 2004.
13. Banerjee, S. and Lavie, A. “METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments.” *ACL Workshop*, 2005.
14. Popović, M. “chrF: character n-gram F-score for automatic MT evaluation.” *WMT*, 2015.
15. Zhang, T. et al. “BERTScore: Evaluating Text Generation with BERT.” *ICLR*, 2020.